

JKP Factor Replication: Mapping Guide for Code Agent

Volatility-Managed Portfolios Thesis — Part II Implementation Brief

1. Purpose

This document instructs a Claude Code agent on how to adapt the JKP GitHub repository (github.com/bkelly-lab/jkp-data) to replicate nine specific factors used in Part I of the thesis at the **individual stock level**. Stock-level weights $w_{i,k,t}$ are required for each factor k and stock i so that trades can be **netted across factors** before transaction costs are applied, following DeMiguel et al. (2024). Simply downloading aggregate factor returns (e.g. from Kenneth French's library) is insufficient — the portfolio construction must be reproduced from the bottom up.

2. JKP Repository Overview

The repository contains two main scripts:

code/main.py — Pulls raw data from WRDS (CRSP + Compustat) and constructs stock-level characteristics for all 153 signals. Output: stock × month panel with characteristics and returns.

code/portfolio.py — Takes the stock-level panel and builds long-short factor returns using JKP's standard *univariate tercile sort* construction: NYSE 30/70 breakpoints, capped value-weighting, monthly rebalancing.

The key insight is that **main.py already computes all raw stock-level characteristics needed** for all 9 factors. The work for this thesis is to write an additional portfolio construction module — call it **code/thesis_factors.py** — that applies the correct factor-specific sorting procedures on top of the existing stock data, rather than using JKP's generic tercile sort.

3. Factor Classification

The nine factors fall into three groups based on implementation effort:

Group	Factors	Action
A — Direct use	ROE, IA, BAB	Run portfolio.py as-is; JKP construction matches target exactly.
B — Same characteristic, different sort	MOM (UMD), RMW, CMA	Raw characteristic available from main.py; write FF 2x3 double-sort in thesis_factors.py.
C — Construct from raw CRSP data	MktRF, SMB, HML	Characteristic available (me, be_me); construct value-weighted or FF-sorted portfolio in thesis_factors.py.

4. Detailed Factor Specifications

4A. Group A — Direct Use from portfolio.py

These three factors are replicated exactly by JKP's existing construction. Run **portfolio.py** and extract the relevant output columns. No changes needed.

Factor	JKP variable	Reference	Notes
ROE	roe	Hou, Xue, Zhang (2015)	Monthly triple 2x3x3 sort on size x I/A x ROE. Quarterly earnings (IBQ). Exact HXZ construction.
IA	inv	Hou, Xue, Zhang (2015)	Monthly triple 2x3x3 sort on size x I/A x ROE. Annual asset growth (AT). Exact HXZ construction.
BAB	betabab_1260d	Frazzini & Pedersen (2014)	Beta estimated over 1260 days. Long leveraged low-beta / short high-beta. Exact FP construction.

4B. Group B — Same Characteristic, Different Sort (FF 2x3 Double Sort)

The underlying stock-level characteristic is already computed by **main.py**, but JKP applies a univariate tercile sort. The Fama-French construction requires a **2x3 independent double sort** on size and the characteristic, with annual June-end rebalancing (except MOM which rebalances monthly). These must be implemented in **thesis_factors.py**.

Factor	JKP characteristic	Required sort construction	Key difference from JKP
MOM (UMD)	ret_12_1 (Jegadeesh-Titman momentum)	FF 2x3 independent sort: NYSE median (size) x NYSE 30/70 (prior return t-12 to t-2). Monthly rebalancing. Long winner/short loser.	(1) FF skips month t-1 (uses t-12 to t-2); JKP uses t-12 to t-1. (2) FF double-sorts with size; JKP univariate. Use ret_12_1 but reconstruct with t-12 to t-2 window.
RMW	ope_be (Operating profits to book equity)	FF 2x3 independent sort: NYSE median (size) x NYSE 30/70 (operating profitability). Annual June-end rebalancing. Long robust/short weak.	Characteristic definition identical to FF (2015). Only sort differs: JKP univariate tercile monthly; FF double-sort annual. Use ope_be directly.
CMA	at_gr1 (Asset Growth 1yr)	FF 2x3 independent sort: NYSE median (size) x NYSE 30/70 (asset growth). Annual June-end rebalancing. Long conservative/short aggressive.	Characteristic definition identical to FF (2015): $(AT_t - AT_{t-1}) / AT_{t-1}$. Only sort differs. Use at_gr1 directly.

4C. Group C — Construct from Raw CRSP/Compustat Characteristics

These factors require raw market data (prices, shares, book equity) rather than a pre-computed anomaly characteristic. The necessary raw variables are already output by **main.py** as part of the stock-level panel. Portfolio construction must be implemented in **thesis_factors.py**.

Factor	Raw variables needed (from main.py output)	Required construction	Stock-level weight formula
MktRF	me (market equity) ret_exc (excess return)	Value-weighted portfolio of all stocks. No sorting. Subtract risk-free rate. Monthly.	$w_{i,MKT,t} = ME_{i,t} / \sum_j (ME_{j,t})$ Long only; all weights positive.

SMB	me (market equity)	FF 2x3 double sort on size x book-to-market (same sort as HML). SMB = avg(3 small portfolios) - avg(3 big portfolios). Annual June-end rebalancing.	Small leg: +1/3 weight across 3 small portfolios (value-wtd within). Big leg: -1/3 weight across 3 big portfolios (value-wtd within).
HML	me (market equity) be_me (book-to-market) be (book equity)	FF 2x3 independent sort: NYSE median (size) x NYSE 30/70 (B/M). Annual June-end rebalancing using Dec fiscal-year B/M. Long high-B/M / short low-B/M.	High B/M leg: +1/2 weight across 2 high-B/M portfolios (value-wtd within). Low B/M leg: -1/2 weight across 2 low-B/M portfolios (value-wtd within).

5. Implementation Instructions for Code Agent

Step 1 — Run the existing pipeline unchanged

Execute `main.py` to generate the full stock-level panel from WRDS. This produces a Parquet file per country-month with all 153 characteristics plus raw market variables (`me`, `ret_exc`, `be`, `be_me`, etc.). Do not modify `main.py`.

Run `portfolio.py` for Group A factors only

Execute `portfolio.py` as-is. Extract factor return series and stock-level weight columns for `roe`, `inv`, and `betabab_1260d`. These are ready to use.

Step 2 — Create `code/thesis_factors.py`

This new module reads the stock-level panel output from `main.py` and implements the exact factor constructions for the remaining 6 factors. It should output a DataFrame indexed by (date, permno) containing columns `w_MktRF`, `w_SMB`, `w_HML`, `w_MOM`, `w_RMW`, `w_CMA` representing each stock's weight in each factor.

Key variables available in `main.py` output panel:

Variable	Description	Used for
<code>me</code>	Market equity (price x shares)	MktRF weights; size breakpoints for all FF sorts
<code>be_me</code>	Book-to-market equity	HML sort; SMB auxiliary sort
<code>be</code>	Book equity	HML; SMB
<code>ope_be</code>	Operating profits / book equity	RMW sort
<code>at_gr1</code>	Asset growth 1yr	CMA sort
<code>ret_12_1</code>	Prior 12-month return (no skip)	MOM — reconstruct as <code>ret_12_2</code> (skip t-1)
<code>ret_exc</code>	Excess stock return	All factor return calculations
<code>excntry</code>	Exchange country code	Filter to USA for US-only replication
<code>exch_main</code>	Main exchange indicator	Filter to NYSE/AMEX/NASDAQ only
<code>common</code>	Common stock indicator	Exclude REITs, ADRs etc.

Step 3 — FF Double-Sort Helper Function

All Group B and C factors (except MktRF) require the same FF 2x3 sort engine. Implement a reusable function with the following signature and logic:

```
def ff_double_sort(df, char_col, rebalance='annual'):
    # 1. At each rebalancing date:
    # a. Compute NYSE median of me -> size breakpoint
    # b. Compute NYSE 30th/70th pctile of char_col -> char breakpoints
    # c. Assign each stock to 2 size bins x 3 char bins = 6 portfolios
    # 2. Hold positions from July(t) to June(t+1) for annual, or 1 month for monthly
    # 3. Value-weight within each of the 6 portfolios using me
    # 4. Return stock-level weights w_{i,t} with sign (+/-) applied
```

Step 4 — MOM Skip-Month Correction

The main.py panel contains ret_12_1 (cumulative return t-12 to t-1). For the FF MOM factor, recompute the sorting variable as the cumulative return from t-12 to t-2 (i.e. exclude the most recent month t-1) to replicate the standard skip-month convention. This can be done directly from the ret_exc column in the panel.

Step 5 — Validation

For each of the 9 factors, compute the aggregate factor return as the weighted average of stock returns using the constructed weights, and compare the time-series correlation with the benchmark series (French library for MktRF/SMB/HML/MOM/RMW/CMA; q-factor database for ROE/IA; AQR for BAB). Target correlation > 0.97 for all factors.

6. Required Output Format for Netting Analysis

The final output of thesis_factors.py should be a single DataFrame with the following structure, saved as **data/processed/thesis_factor_weights.parquet**:

```
Columns: date, permno, w_MktRF, w_SMB, w_HML, w_MOM, w_RMW, w_CMA, w_ROE, w_IA, w_BAB
Index: one row per stock-month (only stocks with non-zero weight in at least one factor)
Units: portfolio weights (sum of long leg = +1, short leg = -1 for each factor)
```

These weights feed directly into the netting calculation: for each stock i and month t , the *net trade* when volatility-scaling all factors simultaneously is:

$$\text{Net_trade}_{\{i,t\}} = \sum_k [w_{\{i,k,t\}} * (c / \sigma^2_{\{k,t\}}) - w_{\{i,k,t-1\}} * (c / \sigma^2_{\{k,t-1\}})]$$

where $\sigma^2_{\{k,t\}}$ is the realized variance of factor k at time t and c is the scaling constant chosen to match unconditional volatility of each factor.

7. Summary Checklist

- ✓ Run main.py on WRDS — generates stock-level panel with all required characteristics
- ✓ Run portfolio.py — extract ROE, IA, BAB factor weights directly (Group A, no changes needed)
- Create thesis_factors.py — implement ff_double_sort() helper function

- MOM: use `ret_12_1` but recompute sorting variable skipping month t-1 (`ret_12_2`)
- RMW: apply `ff_double_sort()` on `ope_be` with annual June rebalancing
- CMA: apply `ff_double_sort()` on `at_gr1` with annual June rebalancing
- MktRF: value-weight all stocks by `me`, no sorting required
- SMB: by-product of HML double sort — `avg(3 small)` minus `avg(3 big)`
- HML: `ff_double_sort()` on `be_me` with annual June rebalancing
- ✓ Validate all 9 factors against benchmark series (target correlation > 0.97)
- ✓ Output `thesis_factor_weights.parquet` with `(date, permno, w_k)` columns